

Day 9 Hands-on Assignment

Optimizing and Recovering an Oracle Database

John Michael Bilbao

Techstart

August 28, 2026

What this covers

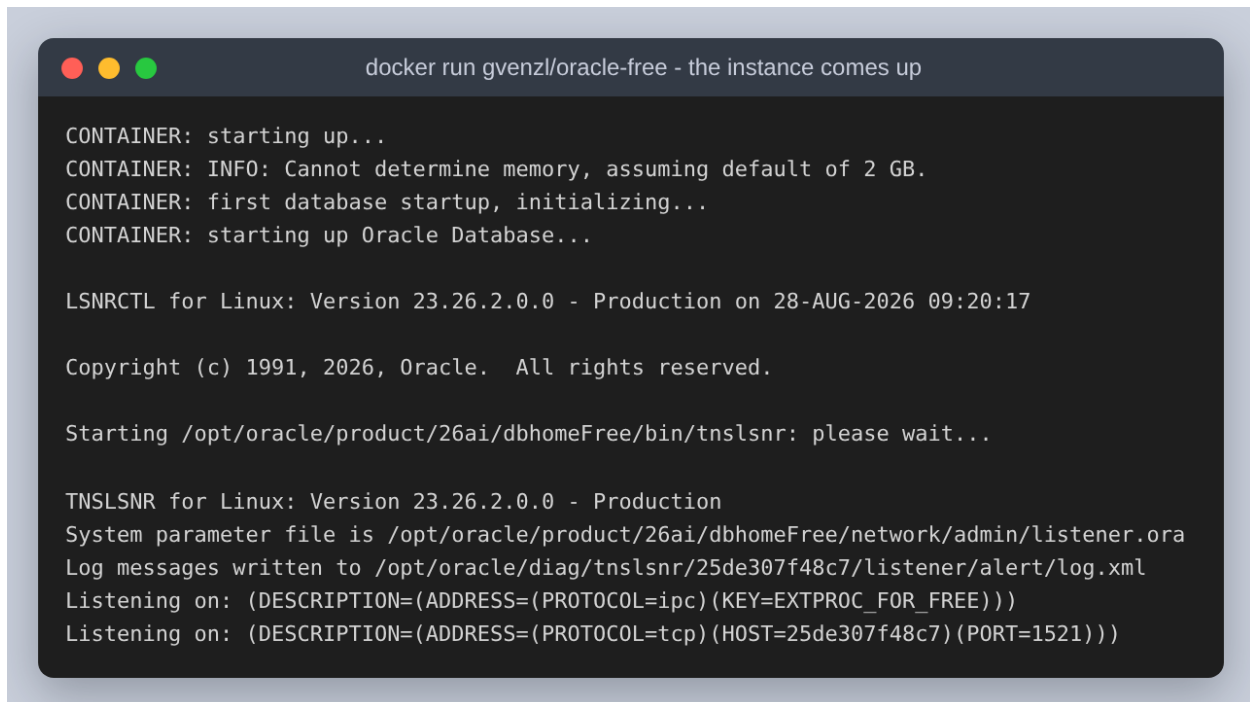
This assignment builds an Oracle database, measures a query against it, indexes the query, then destroys a table and brings it back from an RMAN backup. Everything in it ran against a real instance: -----, started from the gvenzl/oracle-free container image on eight cores with a 1.5 GB SGA.

Every figure is output captured from that instance. The capture files live in results/, scripts/make_figures.py renders those files into the figures below, and none of it is retyped by hand, so no figure can show something the database did not do. The numbers quoted in the text are parsed from the same files.

The short version of what was found: the index the assignment asks for does not speed up the query the assignment asks for, and the execution plans show exactly why. It speeds up a neighbouring query by 10.7x, and a two-column version of it makes the original query read 43 percent fewer blocks while still taking longer on the clock. All three results are below with the plans and the timings that produced them.

Introduction: creating the database

The instance runs in a container, which is the closest thing to a fresh install that can be repeated exactly. `scripts/start_db.sh` creates it and waits for it to open; the container's own startup narration is the record of the creation.

A terminal window with a dark background and light gray text. The title bar at the top shows three colored circles (red, yellow, green) on the left and the text "docker run gvenzl/oracle-free - the instance comes up" on the right. The terminal output shows the container starting up, initializing the database, and starting the TNS listener on port 1521.

```
CONTAINER: starting up...
CONTAINER: INFO: Cannot determine memory, assuming default of 2 GB.
CONTAINER: first database startup, initializing...
CONTAINER: starting up Oracle Database...

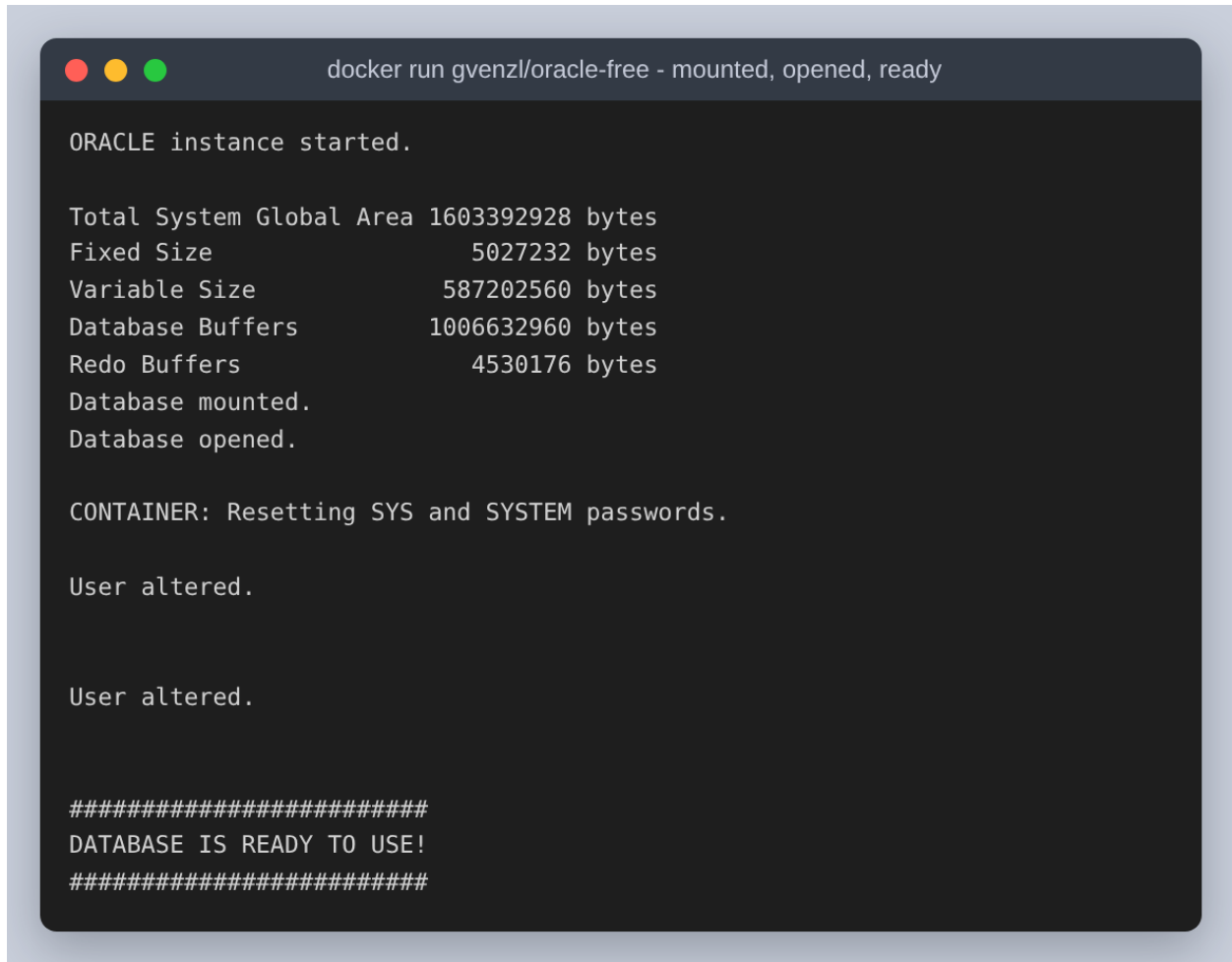
LSNRCTL for Linux: Version 23.26.2.0.0 - Production on 28-AUG-2026 09:20:17

Copyright (c) 1991, 2026, Oracle. All rights reserved.

Starting /opt/oracle/product/26ai/dbhomeFree/bin/tnslsnr: please wait...

TNSLSNR for Linux: Version 23.26.2.0.0 - Production
System parameter file is /opt/oracle/product/26ai/dbhomeFree/network/admin/listener.ora
Log messages written to /opt/oracle/diag/tnslsnr/25de307f48c7/listener/alert/log.xml
Listening on: (DESCRIPTION=(ADDRESS=(PROTOCOL=ipc) (KEY=EXTPROC_FOR_FREE)))
Listening on: (DESCRIPTION=(ADDRESS=(PROTOCOL=tcp) (HOST=25de307f48c7) (PORT=1521)))
```

Figure 1: The instance being created: the container initialises the database and starts the TNS listener on port 1521.



```
docker run gvenzl/oracle-free - mounted, opened, ready

ORACLE instance started.

Total System Global Area 1603392928 bytes
Fixed Size                  5027232 bytes
Variable Size              587202560 bytes
Database Buffers          1006632960 bytes
Redo Buffers               4530176 bytes
Database mounted.
Database opened.

CONTAINER: Resetting SYS and SYSTEM passwords.

User altered.

User altered.

#####
DATABASE IS READY TO USE!
#####
```

Figure 2: The same startup finishing: the SGA is allocated, the database is mounted and opened, and the container reports it ready to use.

Oracle 23ai is multitenant, so what came up is not one database but a container database, FREE, holding a seed and one pluggable database, FREEPDB1. That distinction matters twice later on: the schema is built inside FREEPDB1, while ARCHIVELOG mode and the RMAN backup are properties of the container database around it.

```
sqlplus / as sysdba - what the instance is

=== version ===

BANNER_FULL
-----
Oracle AI Database 26ai Free Release 23.26.2.0.0 - Develop, Learn, and Run for
Free
Version 23.26.2.0.0

=== the container database ===

NAME          CDB OPEN_MODE  LOG_MODE
-----
FREE          YES  READ WRITE   NOARCHIVELOG
```

Figure 3: The version and the container database. Note LOG_MODE: the image ships in NOARCHIVELOG, which Step 3 has to change before it can back anything up usefully.

```
sqlplus / as sysdba - the instance, its PDBs and its memory

=== instance ===

INSTANCE_NAME   HOST_NAME        STATUS           DATABASE_STATUS
-----
FREE            25de307f48c7     OPEN            ACTIVE

=== pluggable databases ===

  CON_ID PDB_NAME        OPEN_MODE  RES
-----
      2 PDB$SEED        READ ONLY  NO
      3 FREEPDB1    READ WRITE NO

=== memory and file layout ===

PARAMETER          VALUE
-----
db_block_size      8192
db_create_file_dest
db_recovery_file_dest
db_recovery_file_dest_size 0
pga_aggregate_target 536870912
sga_target          1610612736
undo_management     AUTO
```

Figure 4: The instance, the two pluggable databases inside it, and the memory and file settings the rest of the assignment runs under.

Step 1: creating the tables and the index

departments is created first, because employees.department_id references it and a foreign key cannot point at a table that does not exist yet. Both tables get a primary key, and employees gets a check constraint on salary as well, so that the recovery in Step 3 has something to prove came back besides the rows.

```
• 02_schema.sql

32 PROMPT === departments ===
33 -- departments is created first: employees.department_id references it, and a
34 -- foreign key cannot point at a table that does not exist yet.
35 CREATE TABLE hr_day9.departments (
36     department_id NUMBER(4) NOT NULL,
37     department_name VARCHAR2(40) NOT NULL,
38     CONSTRAINT departments_pk PRIMARY KEY (department_id)
39 );

sql/02_schema.sql PL/SQL Spaces: 4 UTF-8
```

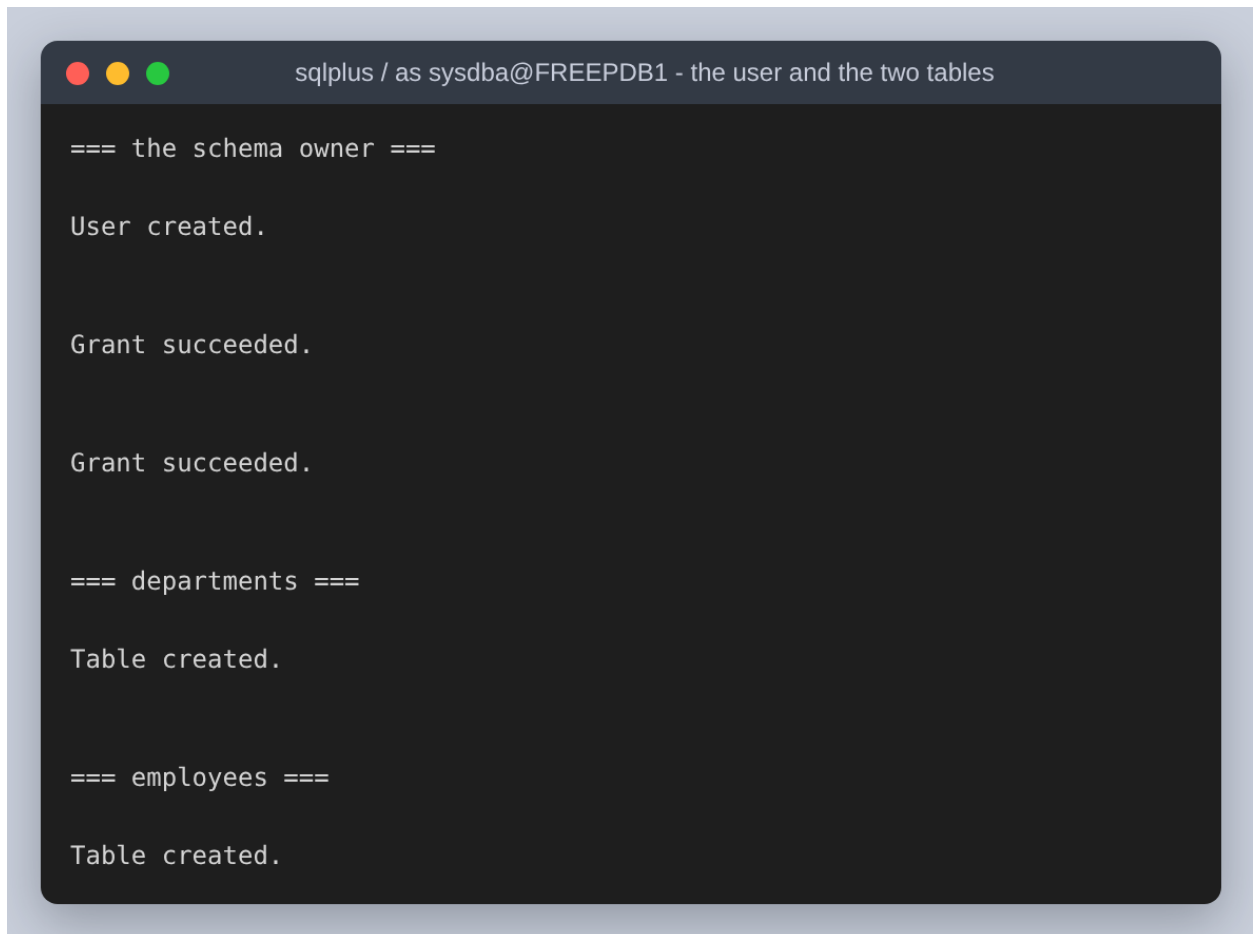
Figure 5: departments: the parent table, created first.

```
• 02_schema.sql

42 PROMPT === employees ===
43 CREATE TABLE hr_day9.employees (
44     employee_id NUMBER(8) NOT NULL,
45     name VARCHAR2(60) NOT NULL,
46     department_id NUMBER(4),
47     salary NUMBER(10, 2) NOT NULL,
48     CONSTRAINT employees_pk PRIMARY KEY (employee_id),
49     CONSTRAINT employees_dept_fk FOREIGN KEY (department_id)
50     REFERENCES hr_day9.departments (department_id),
51     CONSTRAINT employees_salary_ck CHECK (salary > 0)
);

sql/02_schema.sql PL/SQL Spaces: 4 UTF-8
```

Figure 6: employees: primary key, the foreign key to departments, and a check constraint that a salary has to be positive.



```
sqlplus / as sysdba@FREEPDB1 - the user and the two tables

=== the schema owner ===

User created.

Grant succeeded.

Grant succeeded.

=== departments ===

Table created.

=== employees ===

Table created.
```

Figure 7: The schema owner and both tables being created. HR_DAY9 is an ordinary application user; SELECT_CATALOG_ROLE is granted only so that it can read execution plans out of v\$sql_plan in Step 2.

Ten departments and twelve employees are inserted, which is two more employees than the assignment asks for and lets two departments hold more than one person, so the averages in Step 2 are averages of something rather than of a single row.

```
sqlplus / as sysdba@FREEPDB1 - ten departments, twelve employees

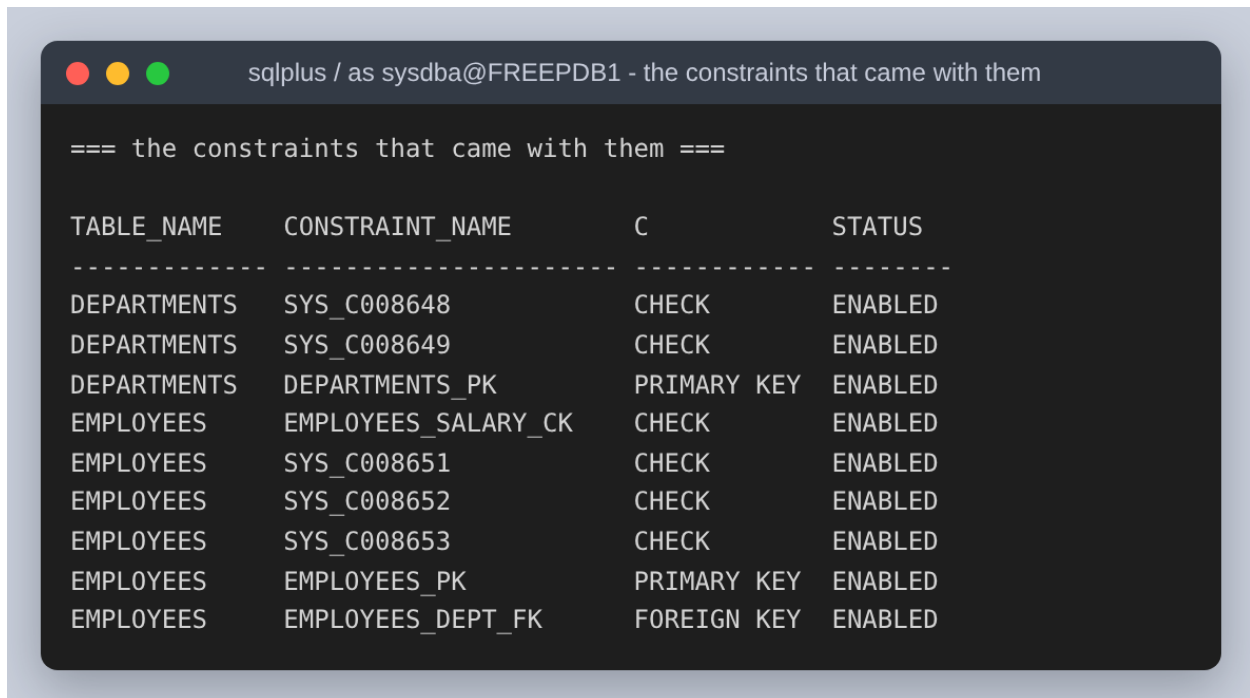
=== what is in the two tables ===

TABLE_NAME      ROWS_LOADED
-----
DEPARTMENTS      10
EMPLOYEES        12

=== the employees, joined to their department ===

EMPLOYEE_ID NAME                DEPARTMENT_NAME      SALARY
-----
1 Amelia Reyes      Executive             185,000.00
2 Bernard Cruz      Finance               96,500.00
3 Corazon Vidal     Finance               88,250.00
4 Diego Almonte     Human Resources       67,400.00
5 Elena Marquez     Information Technology 112,000.00
6 Fidel Ocampo      Information Technology 104,750.00
7 Grace Bautista    Operations            73,900.00
8 Hector Salcedo    Retail Banking        58,300.00
9 Imelda Fajardo    Treasury              121,600.00
10 Joaquin Tolentino Risk and Compliance   99,800.00
11 Karina Delgado   Internal Audit        84,150.00
12 Lorenzo Pineda   Customer Service      61,250.00
```

Figure 8: The row counts after the inserts, and every employee joined to the department that the foreign key points at.



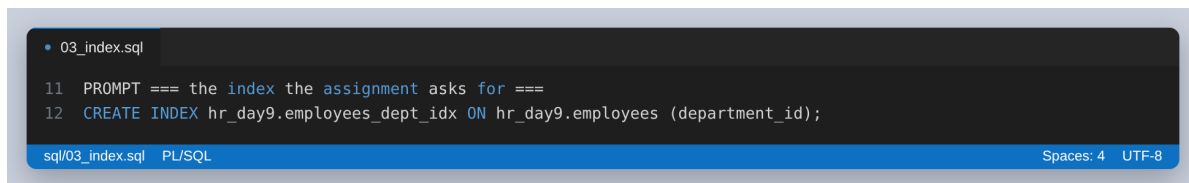
sqlplus / as sysdba@FREEPDB1 - the constraints that came with them

=== the constraints that came with them ===

TABLE_NAME	CONSTRAINT_NAME	C	STATUS
DEPARTMENTS	SYS_C008648	CHECK	ENABLED
DEPARTMENTS	SYS_C008649	CHECK	ENABLED
DEPARTMENTS	DEPARTMENTS_PK	PRIMARY KEY	ENABLED
EMPLOYEES	EMPLOYEES_SALARY_CK	CHECK	ENABLED
EMPLOYEES	SYS_C008651	CHECK	ENABLED
EMPLOYEES	SYS_C008652	CHECK	ENABLED
EMPLOYEES	SYS_C008653	CHECK	ENABLED
EMPLOYEES	EMPLOYEES_PK	PRIMARY KEY	ENABLED
EMPLOYEES	EMPLOYEES_DEPT_FK	FOREIGN KEY	ENABLED

Figure 9: The constraints that came with the two tables. The SYS_C-prefixed checks are the NOT NULL declarations, which Oracle stores as check constraints with generated names.

The index the assignment asks for is on employees.department_id. It is worth being clear that it is the third index in the schema and not the first: Oracle built a unique index behind each primary key automatically, and those exist whether they are wanted or not. This one is the only index so far that is a deliberate choice, and therefore the only one whose value is worth arguing about.



```

11 PROMPT === the index the assignment asks for ===
12 CREATE INDEX hr_day9.employees_dept_idx ON hr_day9.employees (department_id);
  
```

sql/03_index.sql PL/SQL Spaces: 4 UTF-8

Figure 10: The index on employees.department_id.

```
sqlplus / as sysdba@FREEPDB1 - the index on employees.department_id

=== the index the assignment asks for ===

Index created.

=== every index on the two tables now ===

TABLE_NAME      INDEX_NAME      INDEX_TYPE  UNIQUENESS  COLUMNS
-----
DEPARTMENTS     DEPARTMENTS_PK  NORMAL      UNIQUE      DEPARTMENT_ID
EMPLOYEES        EMPLOYEES_DEPT_IDX  NORMAL      NONUNIQUE   DEPARTMENT_ID
EMPLOYEES        EMPLOYEES_PK     NORMAL      UNIQUE      EMPLOYEE_ID

=== and what each one costs on disk ===

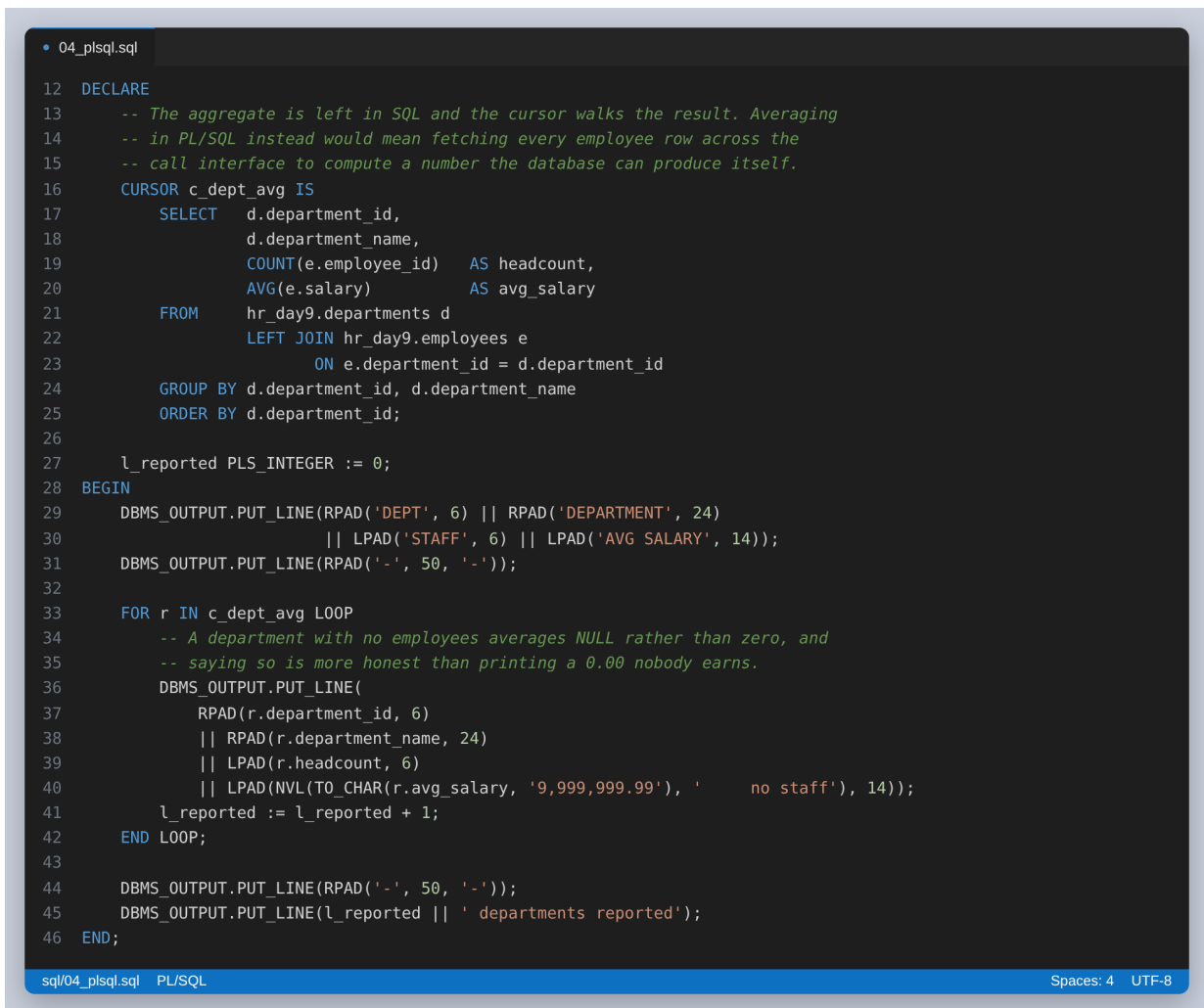
SEGMENT_NAME      SEGMENT_TYPE      KB      BLOCKS
-----
DEPARTMENTS_PK     INDEX              64       8
EMPLOYEES_DEPT_IDX  INDEX              64       8
EMPLOYEES_PK       INDEX              64       8
DEPARTMENTS        TABLE             64       8
EMPLOYEES          TABLE             64       8
```

Figure 11: The index being created, all three indexes on the two tables, and what each costs on disk. At twelve rows every segment is one 64 KB extent, so the index costs as much space as the table it indexes.

Step 2: the PL/SQL query

The assignment asks for a PL/SQL query returning the average salary of employees in each department. It is written twice: once as an anonymous block that reports every department, and once as a stored function that SQL can call per department.

One decision inside the block is worth stating, because it is the difference between PL/SQL that helps and PL/SQL that hurts. The average is computed by SQL, in the cursor, and the PL/SQL only walks the result and formats it. Fetching all the employee rows into PL/SQL and averaging them in a loop would produce the same answer while moving every row across the call interface to compute a number the database can produce itself.



```
04_plsql.sql
12 DECLARE
13     -- The aggregate is left in SQL and the cursor walks the result. Averaging
14     -- in PL/SQL instead would mean fetching every employee row across the
15     -- call interface to compute a number the database can produce itself.
16     CURSOR c_dept_avg IS
17         SELECT  d.department_id,
18                 d.department_name,
19                 COUNT(e.employee_id) AS headcount,
20                 AVG(e.salary)       AS avg_salary
21         FROM    hr_day9.departments d
22                LEFT JOIN hr_day9.employees e
23                      ON e.department_id = d.department_id
24         GROUP BY d.department_id, d.department_name
25         ORDER BY d.department_id;
26
27     l_reported PLS_INTEGER := 0;
28 BEGIN
29     DBMS_OUTPUT.PUT_LINE(RPAD('DEPT', 6) || RPAD('DEPARTMENT', 24)
30                          || LPAD('STAFF', 6) || LPAD('AVG SALARY', 14));
31     DBMS_OUTPUT.PUT_LINE(RPAD('-', 50, '-'));
32
33     FOR r IN c_dept_avg LOOP
34         -- A department with no employees averages NULL rather than zero, and
35         -- saying so is more honest than printing a 0.00 nobody earns.
36         DBMS_OUTPUT.PUT_LINE(
37             RPAD(r.department_id, 6)
38             || RPAD(r.department_name, 24)
39             || LPAD(r.headcount, 6)
40             || LPAD(NVL(TO_CHAR(r.avg_salary, '9,999,999.99'), 'no staff'), 14));
41         l_reported := l_reported + 1;
42     END LOOP;
43
44     DBMS_OUTPUT.PUT_LINE(RPAD('-', 50, '-'));
45     DBMS_OUTPUT.PUT_LINE(l_reported || ' departments reported');
46 END;
```

sql/04_plsql.sql PL/SQL Spaces: 4 UTF-8

Figure 12: The anonymous block. The aggregate is left in SQL; the LEFT JOIN means a department with no employees is still reported.

```
sqlplus / as sysdba@FREEPDB1 - the anonymous block's output

=== an anonymous PL/SQL block, one row per department ===
DEPT  DEPARTMENT          STAFF  AVG SALARY
-----
10    Executive             1      185,000.00
20    Finance               2       92,375.00
30    Human Resources       1       67,400.00
40    Information Technology  2     108,375.00
50    Operations            1       73,900.00
60    Retail Banking        1       58,300.00
70    Treasury              1     121,600.00
80    Risk and Compliance   1       99,800.00
90    Internal Audit        1       84,150.00
100   Customer Service       1       61,250.00
-----
10 departments reported
```

Figure 13: The block's output. All ten departments appear, including the eight that hold exactly one person.

```
04_plsql.sql
52 CREATE OR REPLACE FUNCTION hr_day9.dept_avg_salary (
53   p_department_id IN hr_day9.departments.department_id%TYPE
54 ) RETURN NUMBER
55 IS
56   l_avg hr_day9.employees.salary%TYPE;
57 BEGIN
58   SELECT AVG(salary)
59   INTO   l_avg
60   FROM   hr_day9.employees
61   WHERE  department_id = p_department_id;
62
63   RETURN ROUND(l_avg, 2);
64 EXCEPTION
65   -- AVG over no rows returns NULL rather than raising, so NO_DATA_FOUND
66   -- cannot happen here. An invalid department is still worth rejecting
67   -- loudly instead of returning a NULL the caller may read as zero.
68   WHEN OTHERS THEN
69     RAISE_APPLICATION_ERROR(-20001,
70       'cannot average department ' || p_department_id || ': ' || SQLERRM);
71 END dept_avg_salary;

sql/04_plsql.sql  PL/SQL  Spaces: 4  UTF-8
```

Figure 14: The same average as a stored function. AVG over no rows returns NULL rather than raising, so NO_DATA_FOUND cannot happen here, and the comment says so rather than leaving a handler that never fires looking like it matters.

```
sqlplus / as sysdba@FREEPDB1 - the same averages through the stored
function

=== the same thing as a stored function, so SQL can call it too ===
No errors.
```

DEPARTMENT_ID	DEPARTMENT_NAME	AVG_SALARY
10	Executive	185,000.00
20	Finance	92,375.00
30	Human Resources	67,400.00
40	Information Technology	108,375.00
50	Operations	73,900.00
60	Retail Banking	58,300.00
70	Treasury	121,600.00
80	Risk and Compliance	99,800.00
90	Internal Audit	84,150.00
100	Customer Service	61,250.00

Figure 15: The function called once per department from ordinary SQL, giving the same ten averages.

The plans at twelve rows

The assignment asks for the plan before and after optimization. Taken at twelve rows, that comparison has no content, and it is worth showing why before moving on rather than presenting a meaningless pair of plans as a result.

```

sqlplus / as sysdba@FREEPDB1 - Q1 on twelve rows

=== how big the optimizer thinks the tables are ===

TABLE_NAME          NUM_ROWS    BLOCKS  AVG_ROW_LEN
-----
DEPARTMENTS          10           5         17
EMPLOYEES            12           5         25

=== Q1: average salary per department, all twelve rows ===

DEPARTMENT_ID  AVG_SALARY
-----
10             185000
20             92375
30             67400
40            108375
50             73900
60             58300
70            121600
80             99800
90             84150
100            61250

PLAN_TABLE_OUTPUT
-----
SQL_ID  7v0qt6xslkzj4, child number 0
-----
SELECT /*+ GATHER_PLAN_STATISTICS q1_small */      department_id,
AVG(salary) AS avg_salary FROM    hr_day9.employees GROUP BY
department_id ORDER BY department_id

Plan hash value: 2107619104

-----
| Id | Operation          | Name          | Starts | E-Rows | A-Rows | A-Time   | Buffers | OMem | lMem | Used-Mem |
-----
| 0 | SELECT STATEMENT    |               |       1 |       |    10 | 00:00:00.01 |        6 |      |      |          |
| 1 |  SORT GROUP BY      |               |       1 |    10 |    10 | 00:00:00.01 |        6 | 2048 | 2048 | 2048 (0) |
| 2 |   TABLE ACCESS FULL| EMPLOYEES     |       1 |    12 |    12 | 00:00:00.01 |        6 |      |      |          |
-----

```

Figure 16: Q1, the average per department, on twelve rows: a full scan costing six buffer gets. The whole table is five blocks, so no index can save more than a few block reads.

```

sqlplus / as sysdba@FREEPDB1 - Q2 on twelve rows

=== Q2: average salary for one department ===

AVG_SALARY
-----
      185000

PLAN_TABLE_OUTPUT
-----
SQL_ID  gzv072hxgnz4v, child number 0
-----
SELECT /*+ GATHER_PLAN_STATISTICS q2_small */      AVG(salary) AS
avg_salary FROM    hr_day9.employees WHERE  department_id = 10

Plan hash value: 2413904330

-----
| Id | Operation                                | Name                | Starts | E-Rows | A-Rows |   A-Time   | Buffers |
-----+-----+-----+-----+-----+-----+-----+
|  0 | SELECT STATEMENT                        |                     |       1 |         |         | 00:00:00.01 |        2 |
|  1 |   SORT AGGREGATE                        |                     |       1 |         |         | 00:00:00.01 |        2 |
|  2 |    TABLE ACCESS BY INDEX ROWID BATCHED | EMPLOYEES           |       1 |         |         | 00:00:00.01 |        2 |
|* 3 |     INDEX RANGE SCAN                    | EMPLOYEES_DEPT_IDX |       1 |         |         | 00:00:00.01 |        1 |
-----

Predicate Information (identified by operation id):
-----
   3 - access("DEPARTMENT_ID"=10)

```

Figure 17: Q2, the average for one department, on twelve rows. Here the optimizer does pick the index, and reads two blocks instead of six. A saving of four block reads is real and irrelevant.

Both plans are correct and neither is interesting. An index is a trade: a second structure to maintain on every insert, in exchange for reading less on the way in. At five blocks there is nothing to read less of. To say anything about optimization the table has to be big enough for the two plans to cost different amounts.

Making the table big enough for the question to have an answer

200,000 more employees are loaded into the same table, across the same ten departments, with the distribution skewed the way a real company is skewed: one small department and nine large ones. That skew is the point. An index helps when it can rule most of the table out, so the interesting query is the one that asks about the small department.

sqlplus / as sysdba@FREEPDB1 - 200,000 more employees, and the histogram

=== 200,000 more employees ===

200000 rows created.

Elapsed: 00:00:04.15

Commit complete.

Elapsed: 00:00:00.00

=== restate the statistics, with a histogram on department_id ===

TABLE_NAME	NUM_ROWS	BLOCKS	AVG_ROW_LEN
DEPARTMENTS	10	5	17
EMPLOYEES	200,012	1000	30

=== the histogram Oracle built ===

COLUMN_NAME	NUM_DISTINCT	NUM_BUCKETS	HISTOGRAM
DEPARTMENT_ID	10	10	FREQUENCY
EMPLOYEE_ID	200012	1	NONE
NAME	199168	1	NONE
SALARY	196912	1	NONE

Figure 18: The load, and the statistics gathered afterwards. Oracle built a FREQUENCY histogram with one bucket per department on department_id.

The histogram is what makes the rest of Step 2 work. Without one the optimizer assumes the ten departments are the same size and costs a lookup on any single department at a tenth of the table, which is too much to be worth an index. The histogram tells it that department 10 is 0.25 percent of the rows, and that is the difference between a full scan and an index range scan.

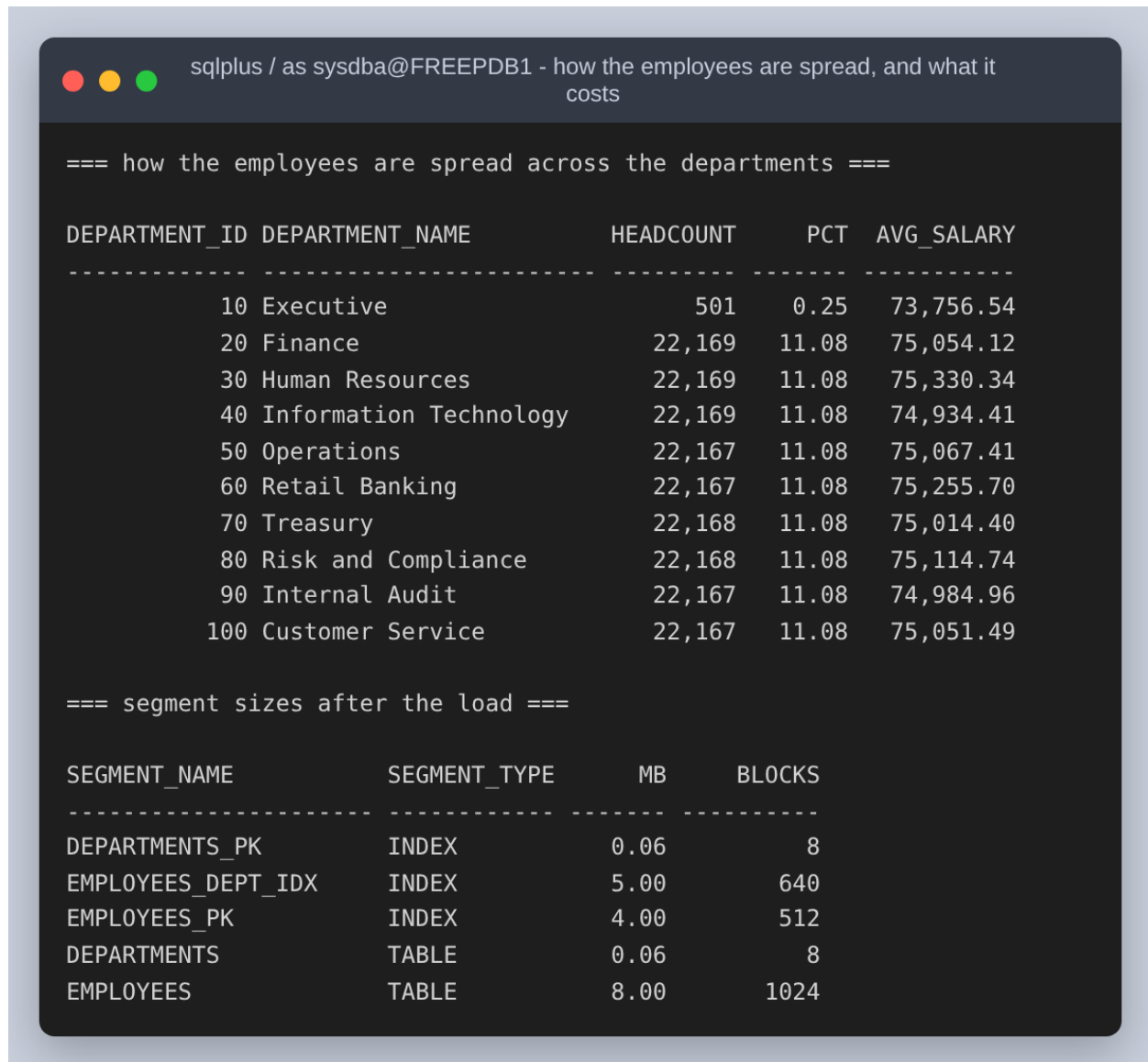


Figure 19: The real distribution: 501 employees in Executive against roughly 22,200 in each of the other nine, and what the segments now cost. The index on department_id alone is already 5 MB against the table's 8 MB.

Before and after, on the same table

To measure the query without the index, the index is made INVISIBLE rather than dropped. Oracle keeps maintaining an invisible index but hides it from the optimizer, so the before and after plans are taken against a byte-for-byte identical table with identical statistics. Dropping and recreating would also rebuild the segment, and then the comparison would have two variables in it instead of one.

```
• 07_explain_before.sql
13 ALTER INDEX hr_day9.employees_dept_idx INVISIBLE;

sql/07_explain_before.sql PL/SQL Spaces: 4 UTF-8
```

Figure 20: One statement is the whole of the before/after apparatus.

```
sqlplus / as sysdba@FREEPDB1 - Q1 with the index invisible

=== the index is still there, and still maintained ===

INDEX_NAME          STATUS  VISIBILITY
-----
EMPLOYEES_DEPT_IDX  VALID   INVISIBLE

=== Q1 before: average salary per department, no usable index ===

DEPARTMENT_ID  AVG_SALARY
-----
10  73756.5426
20  75054.1218
30  75330.3363
40  74934.4121
50  75067.4134
60  75255.6995
70  75014.3957
80  75114.7386
90  74984.9644
100 75051.4896

PLAN_TABLE_OUTPUT
-----
SQL_ID  6myrxws71j43, child number 0
-----
SELECT /*+ GATHER_PLAN_STATISTICS q1_before */      department_id,
AVG(salary) AS avg_salary FROM   hr_day9.employees GROUP BY
department_id ORDER BY department_id

Plan hash value: 2107619104

-----
| Id | Operation          | Name      | Starts | E-Rows | A-Rows | A-Time   | Buffers | OMem | IMem | Used-Mem |
-----
| 0 | SELECT STATEMENT   |           |       1 |        |       10 | 00:00:00.01 | 1004 |      |      |           |
| 1 |  SORT GROUP BY      |           |       1 |    10 |       10 | 00:00:00.01 | 1004 | 2048 | 2048 | 2048 (0) |
| 2 |   TABLE ACCESS FULL| EMPLOYEES |       1 |    200K | 200K | 00:00:00.01 | 1004 |      |      |           |
-----
```

Figure 21: Before, Q1: TABLE ACCESS FULL, 1,006 buffer gets.

```

sqlplus / as sysdba@FREEPDB1 - Q2 with the index invisible

=== Q2 before: average salary for department 10, no usable index ===

AVG_SALARY
-----
73756.5426

PLAN_TABLE_OUTPUT
-----
SQL_ID dx8bd3sug86un, child number 0
-----
SELECT /*+ GATHER_PLAN_STATISTICS q2_before */      AVG(salary) AS
avg_salary FROM   hr_day9.employees WHERE  department_id = 10

Plan hash value: 1756381138

-----
| Id | Operation          | Name          | Starts | E-Rows | A-Rows | A-Time   | Buffers |
-----
|  0 | SELECT STATEMENT    |               |       1 |        |       1 | 00:00:00.01 | 1004 |
|  1 |  SORT AGGREGATE      |               |       1 |        |       1 | 00:00:00.01 | 1004 |
|*  2 |    TABLE ACCESS FULL| EMPLOYEES     |       1 |    501 |    501 | 00:00:00.01 | 1004 |
-----

Predicate Information (identified by operation id):
-----
      2 - filter("DEPARTMENT_ID"=10)

```

Figure 22: Before, Q2: also TABLE ACCESS FULL, 1,004 buffer gets, with the department filter applied to every row it reads.

```

sqlplus / as sysdba@FREEPDB1 - Q1 with the index visible

=== the index is visible to the optimizer again ===

INDEX_NAME          STATUS  VISIBILITY
-----
EMPLOYEES_DEPT_IDX  VALID   VISIBLE

=== Q1 after: average salary per department, index available ===

DEPARTMENT_ID  AVG_SALARY
-----
10  73756.5426
20  75054.1218
30  75330.3363
40  74934.4121
50  75067.4134
60  75255.6995
70  75014.3957
80  75114.7386
90  74984.9644
100 75051.4896

PLAN_TABLE_OUTPUT
-----
SQL_ID 879jhqmw86tbn, child number 0
-----
SELECT /*+ GATHER_PLAN_STATISTICS q1_after */      department_id,
AVG(salary) AS avg_salary FROM   hr_day9.employees GROUP BY
department_id ORDER BY department_id

Plan hash value: 2107619104

-----
| Id | Operation          | Name          | Starts | E-Rows | A-Rows | A-Time   | Buffers | OMem | lMem | Used-Mem |
-----
|  0 | SELECT STATEMENT    |               |       1 |        |       1 | 00:00:00.01 | 1004 |      |      |          |
|  1 |  SORT GROUP BY      |               |       1 |    10 |    10 | 00:00:00.01 | 1004 | 2048 | 2048 | 2048 (0) |
|  2 |    TABLE ACCESS FULL| EMPLOYEES     |       1 |   200K |   200K | 00:00:00.01 | 1004 |      |      |          |
-----

```

Figure 23: After, Q1: the index is visible and the plan has not changed. Same operation, same plan hash value, same 1,006 buffer gets.

```

sqlplus / as sysdba@FREEPDB1 - Q2 with the index visible

=== Q2 after: average salary for department 10, index available ===

AVG_SALARY
-----
73756.5426

PLAN_TABLE_OUTPUT
-----
SQL_ID b5dgh4b0u5jp0, child number 0
-----
SELECT /*+ GATHER_PLAN_STATISTICS q2_after */      AVG(salary) AS
avg_salary FROM    hr_day9.employees WHERE  department_id = 10

Plan hash value: 2413904330

-----
| Id | Operation                                | Name                | Starts | E-Rows | A-Rows |   A-Time   | Buffers |
-----+-----+-----+-----+-----+-----+-----+-----+
|  0 | SELECT STATEMENT                        |                     |      1 |        |       1 | 00:00:00.01 |    505 |
|  1 |   SORT AGGREGATE                       |                     |      1 |        |       1 | 00:00:00.01 |    505 |
|  2 |    TABLE ACCESS BY INDEX ROWID BATCHED | EMPLOYEES           |      1 |    501 |    501 | 00:00:00.01 |    505 |
|*  3 |      INDEX RANGE SCAN                   | EMPLOYEES_DEPT_IDX |      1 |    501 |    501 | 00:00:00.01 |      4 |
-----

Predicate Information (identified by operation id):
-----
   3 - access("DEPARTMENT_ID"=10)

```

Figure 24: After, Q2: INDEX RANGE SCAN feeding TABLE ACCESS BY INDEX ROWID BATCHED. The index itself costs four gets; the row lookups bring the total to 505.

Query	Before	After	Buffer gets
Q1 - average per department	TABLE ACCESS FULL	TABLE ACCESS FULL	1,006 to 1,006
Q2 - average for department 10	TABLE ACCESS FULL	INDEX RANGE SCAN	1,004 to 505

Q1 is unchanged, and that is not a failure of the index or of the optimizer. Q1 averages the salary of every employee, so it has to read every employee. An index on department_id can find rows by department, but it does not contain the salaries, so any plan built on it would still have to visit all 200,012 table rows to get them, one at a time through a rowid lookup, which is strictly more work than reading the table straight through.

That is a claim about cost, so it is worth checking rather than asserting. Hinting the query to use the index anyway shows what the optimizer decided:

```

sqlplus / as sysdba@FREEPDB1 - forcing the index on Q1 with a hint

=== what it costs to force the index on Q1 anyway ===

DEPARTMENT_ID AVG SALARY
-----
10 73756.5426
20 75054.1218
30 75330.3363
40 74934.4121
50 75067.4134
60 75255.6995
70 75014.3957
80 75114.7386
90 74984.9644
100 75051.4896

PLAN_TABLE_OUTPUT
-----
SQL_ID 4q0g0w7guv5hk, child number 0
-----
SELECT /*+ GATHER_PLAN_STATISTICS INDEX(e employees_dept_idx) q1_forced
*/
   department_id, AVG(salary) AS avg_salary FROM
hr_day9.employees e GROUP BY department_id ORDER BY department_id

Plan hash value: 2107619104

-----
| Id | Operation          | Name                | Starts | E-Rows | A-Rows |   A-Time   | Buffers |  0Mem |  1Mem | Used-Mem |
-----
|  0 | SELECT STATEMENT    |                     |       1 |        |    10 | 00:00:00.01 |    1004 |       |       |          |
|  1 |  SORT GROUP BY      |                     |       1 |    10 |    10 | 00:00:00.01 |    1004 |    2048 |    2048 |    2048 (0) |
|  2 |   TABLE ACCESS FULL| EMPLOYEES           |       1 |    200K |    200K | 00:00:00.01 |    1004 |       |       |          |
-----

```

Figure 25: An INDEX hint naming the index on Q1. The plan is the full scan again: the hint is not obeyed, because the index cannot produce the columns the query needs and a hint cannot make a plan legal that was not. A hint that is silently dropped looks exactly like a hint that worked, which is a good reason to read the plan rather than trust the hint.

The optimisation that does help Q1

If the problem with the single-column index is that it does not hold the salaries, the fix is an index that does. An index on (department_id, salary) holds both columns Q1 reads, so the aggregate can be answered from the index and the table never has to be opened. The index is smaller than the table, because it holds two columns instead of four, and reading less is the whole of the improvement.

```

• 09_covering.sql

14 PROMPT === a covering index for Q1 ===
15 CREATE INDEX hr_day9.employees_dept_sal_idx
16     ON hr_day9.employees (department_id, salary);

sql/09_covering.sql  PL/SQL                                     Spaces: 4  UTF-8

```

Figure 26: The covering index. The comment records why it is allowed to answer a query over the whole table.

That last point is a real constraint and not a detail. A B-tree index leaves a row out only when every indexed column is NULL, so an index on a nullable column alone cannot be trusted to contain every row. Here salary is NOT NULL, which guarantees an entry for every employee, and that guarantee is what lets the optimizer answer a GROUP BY over the whole table from the index. Had salary been nullable, this optimisation would not have been available.

```

sqlplus / as sysdba@FREEPDB1 - the covering index, and its size

=== a covering index for Q1 ===

Index created.

=== how much smaller the index is than the table ===

SEGMENT_NAME                SEGMENT_TYPE      MB      BLOCKS
-----
EMPLOYEES                    TABLE            8.00     1024
EMPLOYEES_DEPT_SAL_IDX       INDEX             5.00      640
EMPLOYEES_DEPT_IDX           INDEX             5.00      640

```

Figure 27: The covering index being created, and the sizes side by side.

```

sqlplus / as sysdba@FREEPDB1 - Q1 answered from the covering index alone

=== Q1 with the covering index available ===

DEPARTMENT_ID AVG_SALARY
-----
10 73756.5426
20 75054.1218
30 75330.3363
40 74934.4121
50 75067.4134
60 75255.6995
70 75014.3957
80 75114.7386
90 74984.9644
100 75051.4896

PLAN_TABLE_OUTPUT
-----
SQL_ID 7jfgd2v1v2c9p, child number 0
-----
SELECT /*+ GATHER_PLAN_STATISTICS q1_covering */      department_id,
AVG(salary) AS avg_salary FROM    hr_day9.employees GROUP BY
department_id ORDER BY department_id

Plan hash value: 724969776

-----
| Id | Operation                | Name                | Starts | E-Rows | A-Rows | A-Time   | Buffers | OMem | IMem | Used-Mem |
-----
| 0 | SELECT STATEMENT          |                     | 1      |        | 10     | 00:00:00.01 | 567     |      |      |          |
| 1 | SORT GROUP BY             |                     | 1      | 10     | 10     | 00:00:00.01 | 567     | 2048 | 2048 | 2048 (0) |
| 2 | INDEX FAST FULL SCAN      | EMPLOYEES_DEPT_SAL_IDX | 1      | 200K   | 200K   | 00:00:00.01 | 567     |      |      |          |
-----

```

Figure 28: Q1 answered by INDEX FAST FULL SCAN with no table access at all: 569 buffer gets against 1,006, a 43 percent reduction.

```

sqlplus / as sysdba@FREEPDB1 - Q2 answered from the covering index alone

=== Q2 with the covering index available ===

AVG_SALARY
-----
73756.5426

PLAN_TABLE_OUTPUT
-----
SQL_ID gln3xw9923dsh, child number 0
-----
SELECT /*+ GATHER_PLAN_STATISTICS q2_covering */      AVG(salary) AS
avg_salary FROM   hr_day9.employees WHERE  department_id = 10

Plan hash value: 1309247300

-----
| Id | Operation          | Name                | Starts | E-Rows | A-Rows | A-Time   | Buffers |
-----
|  0 | SELECT STATEMENT    |                     |       1 |        |       1 | 00:00:00.01 |      4 |
|  1 |  SORT AGGREGATE     |                     |       1 |        |       1 | 00:00:00.01 |      4 |
|*  2 |    INDEX RANGE SCAN | EMPLOYEES_DEPT_SAL_IDX |       1 |      501 |      501 | 00:00:00.01 |      4 |
-----

Predicate Information (identified by operation id):
-----
      2 - access("DEPARTMENT_ID"=10)

=== which indexes the optimizer actually used ===

```

Figure 29: Q2 with the same index: 4 buffer gets. The index range scan now finds the salaries in the index beside the department, so the 501 rowid lookups into the table disappear.

Measured, rather than only planned

Plans predict cost; they do not prove it. Each configuration was therefore run in a loop, Q1 fifty times and Q2 five hundred times, with the logical reads read back out of v\$sqlstats per cursor.

```

sqlplus / as sysdba@FREEPDB1 - 50 runs of Q1 and 500 of Q2 per configuration

Q  INDEXES                RUNS      MS / RUN    GETS / RUN
-----
Q1  no index                50        9.600      1,006
Q2  no index               500        2.140      1,004
Q1  department_id           50        9.400      1,006
Q2  department_id           500        0.200       505
Q1  covering                50       13.000       569
Q2  covering                500        0.020        4

-----
Q1 = average salary per department (whole table)
Q2 = average salary for department 10 only

```

Figure 30: Fifty runs of Q1 and five hundred of Q2 under each of the three index configurations.

	No index	department_id	(department_id, salary)
Q1 buffer gets	1,006	1,006	569
Q1 ms per run	9.600	9.400	13.000
Q2 buffer gets	1,004	505	4
Q2 ms per run	2.140	0.200	0.020

Q2 is the clear win: 10.7x faster with the index the assignment asks for, and 107.0x faster with the covering index, which is what reading four blocks instead of a thousand looks like on the clock.

Q1 is more interesting, because the two measurements disagree. The covering index cuts Q1's logical reads by 43 percent, from 1,006 to 569 blocks, and yet Q1 got slower: 13.000 ms against 9.600 ms. Fewer blocks is not the same as less time. A full table scan is read with multiblock reads, while an index fast full scan of a 200,000-entry index walks more, smaller structures; at this size the table already fits in the buffer cache, so there is no physical I/O for the block saving to save, and only the extra CPU is left. The block reduction is what would pay off on a table too large to cache. Had only the plans been collected, this would have been written up as a straight improvement.

Step 3: backup and recovery

The image ships in NOARCHIVELOG mode, and in that mode the online redo logs are overwritten as they fill. That leaves no record of what changed after a backup, so the only recovery possible is back to the moment of the backup, and only from a backup taken while the database was shut down. ARCHIVELOG mode copies each redo log off before it is reused. It is what makes both a backup of an open database and a recovery to a chosen point in time possible, and this assignment needs both.

```
sqlplus / as sysdba - the bounce through MOUNT that enables ARCHIVELOG

=== before: the log mode as the image ships it ===

LOG_MODE
-----
NOARCHIVELOG

1 row selected.

=== where the archived logs and backups will go ===

System altered.

System altered.

=== a clean bounce through MOUNT, which is where the switch is made ===
Database closed.
Database dismounted.
ORACLE instance shut down.
ORACLE instance started.

Total System Global Area 1603392928 bytes
Fixed Size                  5027232 bytes
Variable Size               587202560 bytes
Database Buffers            1006632960 bytes
Redo Buffers                 4530176 bytes
Database mounted.

Database altered.

Database altered.

Pluggable database altered.
```

Figure 31: The switch to ARCHIVELOG. It is made at MOUNT, so the database is closed and reopened around it; the flash recovery area is pointed somewhere with room first.

```
sqlplus / as sysdba - ARCHIVELOG, and the first archived log

=== after ===

NAME          LOG_MODE      OPEN_MODE
-----
FREE          ARCHIVELOG    READ WRITE

CON_ID PDB_NAME      OPEN_MODE
-----
2 PDB$SEED      READ ONLY
3 FREEPDB1      READ WRITE

=== force a log switch so there is at least one archived log ===

SEQUENCE# FIRST_TIME          STATUS
-----
15 2026-08-28 09:21:03 A

=== flash recovery area usage ===

FILE_TYPE          PCT_USED NUMBER_OF_FILES
-----
ARCHIVED LOG          0.01          1
```

Figure 32: ARCHIVELOG confirmed, both pluggable databases open again, and a forced log switch so that at least one archived log exists before the backup runs.

Before backing anything up, the state being backed up is recorded: the row counts, the total of all 200,012 salaries, a hash fingerprint of the employees table, and the list of objects that will have to come back. Without this, a successful recovery can only be declared, not shown.

```
sqlplus / as sysdba@FREEPDB1 - what the backup is being taken of

=== contents ===

TABLE_NAME          ROW_COUNT          SALARY_TOTAL
-----
DEPARTMENTS          10
EMPLOYEES            200,012      15,018,179,225.12

=== a fingerprint of the employees table ===

FINGERPRINT
-----
430599086091596

=== the objects that must come back ===

OBJECT_NAME          OBJECT_TYPE  STATUS
-----
DEPT_AVG_SALARY      FUNCTION    VALID
DEPARTMENTS_PK       INDEX       VALID
EMPLOYEES_DEPT_IDX   INDEX       VALID
EMPLOYEES_DEPT_SAL_IDX INDEX       VALID
EMPLOYEES_PK         INDEX       VALID
DEPARTMENTS          TABLE      VALID
EMPLOYEES            TABLE      VALID

=== the clock, for the record ===

CLOCK
-----
2026-08-28 09:21:16
```

Figure 33: What the backup is a backup of. The fingerprint sums a hash of every employee row: a total of the salaries alone would not notice two employees swapping values, and a hash of the ordered rows would.

The backup

RMAN is Oracle's own backup tool, and for this it is the one that matters, because it understands the file formats. A filesystem copy of an open database catches datafiles mid-write and produces something that cannot be opened. RMAN reads through the instance, checks every block as it goes, and records what it did in the control file so a later RESTORE knows what exists.

```

• 01_backup.rman
11 SHOW ALL;
12
13 # Two channels because the container has eight cores and the backup is the
14 # slowest step in the assignment.
15 CONFIGURE DEVICE TYPE DISK PARALLELISM 2;
16
17 # With autobackup on, RMAN writes a copy of the control file and the spfile at
18 # the end of every backup. Without it a recovery that has lost the control file
19 # has to be talked through the backup set by hand.
20 CONFIGURE CONTROLFILE AUTOBACKUP ON;
21
22 CONFIGURE RETENTION POLICY TO RECOVERY WINDOW OF 7 DAYS;
23
24 # What is being backed up, before backing it up.
25 REPORT SCHEMA;
26
27 # PLUS ARCHIVELOG archives the current redo log first, includes the archived
28 # logs in the backup, and archives again afterwards. That is what makes the
29 # backup usable on its own: the datafile copies are inconsistent, because the
30 # database was open while they were read, and the redo in the same backup is
31 # what reconciles them.
32 BACKUP AS COMPRESSED BACKUPSET DATABASE PLUS ARCHIVELOG;
33
34 LIST BACKUP SUMMARY;

```

rman/01_backup.rman Shell Script

Spaces: 4 UTF-8

Figure 34: The backup script. *PLUS ARCHIVELOG* is the part that makes the backup usable on its own.

PLUS ARCHIVELOG archives the current redo log first, includes the archived logs in the backup, then archives again afterwards. The datafile copies inside the backup are inconsistent with each other, because the database was open and changing while they were read, and the redo in the same backup is what reconciles them.

```

rman target / - the configuration the backup runs under

CONFIGURE SNAPSHOT CONTROLFILE NAME TO '/opt/oracle/product/26ai/dbhomeFree/dbs/snapcf_FREE.f'; # default

new RMAN configuration parameters:
CONFIGURE DEVICE TYPE DISK PARALLELISM 2 BACKUP TYPE TO BACKUPSET;
new RMAN configuration parameters are successfully stored

new RMAN configuration parameters:
CONFIGURE CONTROLFILE AUTOBACKUP ON;
new RMAN configuration parameters are successfully stored

new RMAN configuration parameters:
CONFIGURE RETENTION POLICY TO RECOVERY WINDOW OF 7 DAYS;
new RMAN configuration parameters are successfully stored

```

Figure 35: The configuration the backup runs under. Control file autobackup is turned on so that a recovery which has also lost the control file does not have to be talked through the backup set by hand.

```
rman target / - REPORT SCHEMA: every file in the backup

Report of database schema for database with db_unique_name FREE

List of Permanent Datafiles
=====
File Size(MB) Tablespace          RB segs Datafile Name
-----
1    930    SYSTEM                YES    /opt/oracle/oradata/FREE/system01.dbf
2    288    PDB$SEED:SYSTEM          NO     /opt/oracle/oradata/FREE/pdbseed/system01.dbf
3    594    SYSAUX                  NO     /opt/oracle/oradata/FREE/sysaux01.dbf
4    373    PDB$SEED:SYSAUX          NO     /opt/oracle/oradata/FREE/pdbseed/sysaux01.dbf
7    10     USERS                   NO     /opt/oracle/oradata/FREE/users01.dbf
18   11     UNDOTBS1                 YES    /opt/oracle/oradata/FREE/undotbs01.dbf
20   11     PDB$SEED:UNDOTBS1        NO     /opt/oracle/oradata/FREE/pdbseed/undotbs01.dbf
21   298    FREEPDB1:SYSTEM          YES    /opt/oracle/oradata/FREE/FREEPDB1/system01.dbf
22   373    FREEPDB1:SYSAUX          NO     /opt/oracle/oradata/FREE/FREEPDB1/sysaux01.dbf
23   41     FREEPDB1:UNDOTBS1        YES    /opt/oracle/oradata/FREE/FREEPDB1/undotbs01.dbf
24   31     FREEPDB1:USERS           NO     /opt/oracle/oradata/FREE/FREEPDB1/users01.dbf
```

Figure 36: REPORT SCHEMA: every datafile in the container database and in both pluggable databases, which is what BACKUP DATABASE will cover.

```
rman target / - BACKUP AS COMPRESSED BACKUPSET DATABASE PLUS ARCHIVELOG

Starting backup at 28-AUG-26
current log archived
RMAN-06908: warning: operation will not run in parallel on the allocated channels
RMAN-06909: warning: parallelism require Enterprise Edition
allocated channel: ORA_DISK_1
channel ORA_DISK_1: SID=52 device type=DISK
channel ORA_DISK_1: starting compressed archived log backup set
channel ORA_DISK_1: specifying archived log(s) in backup set
input archived log thread=1 sequence=15 RECID=1 STAMP=1242465675
input archived log thread=1 sequence=16 RECID=2 STAMP=1242465683
channel ORA_DISK_1: starting piece 1 at 28-AUG-26
channel ORA_DISK_1: finished piece 1 at 28-AUG-26
piece handle=/opt/oracle/oradata/FRA/FREE/backupset/2026_08_28/o1_mf_anncn_TAG20260828T092123_o92nrmsr_.bkp tag=TAG20260828T092123 comment=NONE
channel ORA_DISK_1: backup set complete, elapsed time: 00:00:01
Finished backup at 28-AUG-26

Starting backup at 28-AUG-26
using channel ORA_DISK_1
channel ORA_DISK_1: starting compressed full datafile backup set
channel ORA_DISK_1: specifying datafile(s) in backup set
input datafile file number=00001 name=/opt/oracle/oradata/FREE/system01.dbf
input datafile file number=00003 name=/opt/oracle/oradata/FREE/sysaux01.dbf
input datafile file number=00018 name=/opt/oracle/oradata/FREE/undotbs01.dbf
input datafile file number=00007 name=/opt/oracle/oradata/FREE/users01.dbf
channel ORA_DISK_1: starting piece 1 at 28-AUG-26
channel ORA_DISK_1: finished piece 1 at 28-AUG-26
piece handle=/opt/oracle/oradata/FRA/FREE/backupset/2026_08_28/o1_mf_nnndf_TAG20260828T092124_o92nro55_.bkp tag=TAG20260828T092124 comment=NONE
channel ORA_DISK_1: backup set complete, elapsed time: 00:00:25
channel ORA_DISK_1: starting compressed full datafile backup set
channel ORA_DISK_1: specifying datafile(s) in backup set
input datafile file number=00022 name=/opt/oracle/oradata/FREE/FREEPDB1/sysaux01.dbf
input datafile file number=00021 name=/opt/oracle/oradata/FREE/FREEPDB1/system01.dbf
input datafile file number=00023 name=/opt/oracle/oradata/FREE/FREEPDB1/undotbs01.dbf
input datafile file number=00024 name=/opt/oracle/oradata/FREE/FREEPDB1/users01.dbf
channel ORA_DISK_1: starting piece 1 at 28-AUG-26
channel ORA_DISK_1: finished piece 1 at 28-AUG-26
piece handle=/opt/oracle/oradata/FRA/FREE/53103393CA0F0FE9E0636402000A5830/backupset/2026_08_28/o1_mf_nnndf_TAG20260828T092124_o92nsg5v_.bkp tag=TAG20260828T092124 comment=NONE
channel ORA_DISK_1: backup set complete, elapsed time: 00:00:15
channel ORA_DISK_1: starting compressed full datafile backup set
channel ORA_DISK_1: specifying datafile(s) in backup set
input datafile file number=00004 name=/opt/oracle/oradata/FREE/pdbseed/sysaux01.dbf
input datafile file number=00002 name=/opt/oracle/oradata/FREE/pdbseed/system01.dbf
input datafile file number=00020 name=/opt/oracle/oradata/FREE/pdbseed/undotbs01.dbf
channel ORA_DISK_1: starting piece 1 at 28-AUG-26
channel ORA_DISK_1: finished piece 1 at 28-AUG-26
piece handle=/opt/oracle/oradata/FRA/FREE/50805151DE580D63E06304BF6B64EEE7/backupset/2026_08_28/o1_mf_nnndf_TAG20260828T092124_o92nsx8j_.bkp tag=TAG20260828T092124 comment=NONE
channel ORA_DISK_1: backup set complete, elapsed time: 00:00:15
Finished backup at 28-AUG-26
```

Figure 37: The backup itself: the archived logs, then all fifteen datafiles as compressed backup sets, then the archived logs again.

```
rman target / - the autobackup, and LIST BACKUP SUMMARY

Starting Control File and SPFILE Autobackup at 28-AUG-26
piece handle=/opt/oracle/oradata/FRA/FREE/autobackup/2026_08_28/o1_mf_s_1242465741_o92ntfw0_.bkp comment=NONE
Finished Control File and SPFILE Autobackup at 28-AUG-26

List of Backups
=====
Key       TY LV S Device Type Completion Time #Pieces #Copies Compressed Tag
-----
1         B A A DISK      28-AUG-26      1       1      YES    TAG20260828T092123
2         B F A DISK      28-AUG-26      1       1      YES    TAG20260828T092124
3         B F A DISK      28-AUG-26      1       1      YES    TAG20260828T092124
4         B F A DISK      28-AUG-26      1       1      YES    TAG20260828T092124
5         B A A DISK      28-AUG-26      1       1      YES    TAG20260828T092220
6         B F A DISK      28-AUG-26      1       1      NO     TAG20260828T092221
```

Figure 38: The control file and SPFILE autobackup, and LIST BACKUP SUMMARY showing the six backup sets that resulted.

```
rman target / - VALIDATE DATABASE, and REPORT NEED BACKUP

Starting validate at 28-AUG-26
using channel ORA_DISK_1
channel ORA_DISK_1: starting validation of datafile
channel ORA_DISK_1: specifying datafile(s) for validation
input datafile file number=00001 name=/opt/oracle/oradata/FREE/system01.dbf
input datafile file number=00003 name=/opt/oracle/oradata/FREE/sysaux01.dbf
input datafile file number=00018 name=/opt/oracle/oradata/FREE/undotbs01.dbf
input datafile file number=00007 name=/opt/oracle/oradata/FREE/users01.dbf
channel ORA_DISK_1: validation complete, elapsed time: 00:00:01
List of Datafiles
=====
File Status Marked Corrupt Empty Blocks Blocks Examined High SCN
-----
...
List of Control File and SPFILE
=====
File Type      Status Blocks Failing Blocks Examined
-----
SPFILE         OK      0          2
Control File OK      0         1146
Finished validate at 28-AUG-26

RMAN retention policy will be applied to the command
RMAN retention policy is set to recovery window of 7 days
Report of files that must be backed up to satisfy 7 days recovery window
File Days  Name
-----
Recovery Manager complete.
```

Figure 39: VALIDATE DATABASE reads every block of every file and checks it, so that the recovery is not the first time anyone finds out whether the backup is sound. Zero blocks marked corrupt, and REPORT NEED BACKUP lists nothing outstanding.

One honest note on the configuration. The script asks for PARALLELISM 2, and RMAN answers with RMAN-06908 and RMAN-06909: parallel backup is an Enterprise Edition feature, so Free allocates one channel and says so. The warning is visible in the figures above and in the restore later on. It is left in rather than tidied away, because the setting is harmless and the warning is the evidence for which edition this actually ran on.

The failure

The failure is a dropped employees table, with PURGE. That keyword is deliberate. A plain DROP TABLE moves the segment to the recycle bin, and then the recovery is one FLASHBACK TABLE away and the backup is never touched. PURGE removes that shortcut, which is the point: the assignment asks for a recovery from the backup, so the failure has to be one the backup is the only answer to.

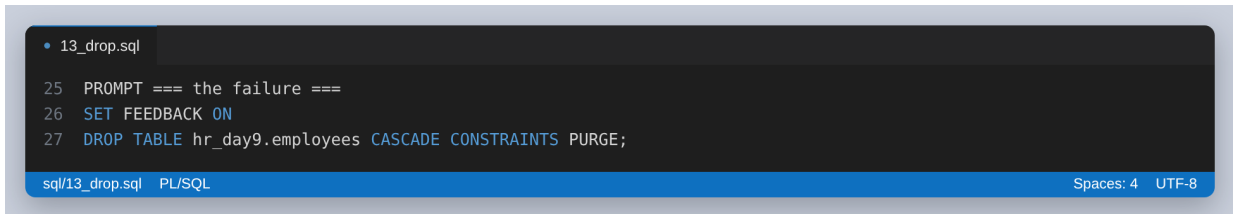
A screenshot of a SQL script editor window. The window has a dark background with a blue header bar. The header bar contains the text 'sql/13_drop.sql' and 'PL/SQL' on the left, and 'Spaces: 4 UTF-8' on the right. The main area of the window displays a SQL script with three lines: line 25 is 'PROMPT === the failure ===', line 26 is 'SET FEEDBACK ON', and line 27 is 'DROP TABLE hr_day9.employees CASCADE CONSTRAINTS PURGE;'. The script is highlighted in a light blue color. The window title bar at the top shows '13_drop.sql'.

Figure 40: The failure, and the SCN read one statement before it.

The SCN is Oracle's own logical clock. Every commit gets a higher one, so recovering until a given SCN means putting the database back the way it was at that instant. It is read one statement before the DROP and passed straight to RMAN by setup.sh, so the number in the recovery command is that number rather than a retyped copy of it.

```
sqlplus / as sysdba@FREEPDB1 - the SCN, then the failure

=== the point the recovery will aim for, read one statement before ===
=== the damage is done ===

RESTORE_POINT
-----
RESTORE_POINT_SCN=2243875

=== the failure ===

Table dropped.

=== it is gone ===

OBJECT_NAME          OBJECT_TYPE  STATUS
-----
DEPT_AVG_SALARY      FUNCTION    INVALID
DEPARTMENTS_PK       INDEX       VALID
DEPARTMENTS          TABLE      VALID

=== nothing in the recycle bin to flash back to ===
```

Figure 41: The restore point, SCN 2243875, then the table dropped. Afterwards HR_DAY9 holds only departments and its index, and the stored function has gone INVALID because the table it reads is gone.

```
sqlplus / as sysdba@FREEPDB1 - neither shortcut can bring it back

=== so the two obvious shortcuts both fail ===
FLASHBACK TABLE hr_day9.employees TO BEFORE DROP
*
ERROR at line 1:
ORA-38305: object not in RECYCLE BIN
Help: https://docs.oracle.com/error-help/db/ora-38305/

SELECT COUNT(*) FROM hr_day9.employees
*
ERROR at line 1:
ORA-00942: table or view "HR_DAY9"."EMPLOYEES" does not exist
Help: https://docs.oracle.com/error-help/db/ora-00942/

=== and the index went with the table ===

INDEX_NAME          TABLE_NAME          STATUS
-----
DEPARTMENTS_PK      DEPARTMENTS          VALID

=== the departments table is untouched, which is what makes this a ===
=== partial loss rather than a lost database                               ===

DEPARTMENTS
-----
10
```

Figure 42: Both shortcuts refusing. FLASHBACK TABLE TO BEFORE DROP fails with ORA-38305 because PURGE left nothing in the recycle bin, and selecting from the table fails with ORA-00942. departments survives untouched, which makes this a partial loss rather than a lost database, and a partial loss is the harder case: the recovery has to bring one table back without rolling the rest of the schema somewhere else.

The recovery

This is a point-in-time recovery of the whole container database to the SCN captured before the DROP. RESTORE copies the datafiles back out of the backup, in the inconsistent state they were read in. RECOVER replays the archived redo logs on top of them up to the SCN, and that is the step that turns the copies into a working database with the employees table in it.

```
• 02_restore.rman

15 RUN {
16     # RESTORE needs the datafiles closed but the control file readable, and
17     # MOUNT is the state where both are true.
18     SHUTDOWN IMMEDIATE;
19     STARTUP MOUNT;
20
21     # SET UNTIL applies to both commands below: it stops RESTORE from picking a
22     # backup taken after the target, and stops RECOVER from applying redo past
23     # it. Setting it once is what keeps the two consistent.
24     SET UNTIL SCN &1;
25
26     # RESTORE copies the datafiles back out of the backup. They come back as
27     # they were during the backup, which is mid-transaction and unusable.
28     RESTORE DATABASE;
29
30     # RECOVER replays the archived redo logs on top of them, up to the SCN, and
31     # that is the step that turns the copies into a consistent database. The
32     # employees table exists again at the end of this line.
33     RECOVER DATABASE;
34
35     ALTER DATABASE OPEN RESETLOGS;
36 }
37
38 LIST INCARNATION OF DATABASE;
```

rman/02_restore.rman Shell Script

Spaces: 4 UTF-8

Figure 43: The recovery script. SET UNTIL is set once and governs both commands, which is what keeps RESTORE from picking a backup taken after the target and RECOVER from applying redo past it.

```
rman target / - SET UNTIL SCN, then RESTORE DATABASE

using target database control file instead of recovery catalog
database closed
database dismounted
Oracle instance shut down

connected to target database (not started)
Oracle instance started
database mounted

Total System Global Area      1603392928 bytes

Fixed Size                      5027232 bytes
Variable Size                  587202560 bytes
Database Buffers               1006632960 bytes
Redo Buffers                    4530176 bytes

executing command: SET until clause (SCN)

Starting restore at 28-AUG-26
RMAN-06908: warning: operation will not run in parallel on the allocated channels
RMAN-06909: warning: parallelism require Enterprise Edition
allocated channel: ORA_DISK_1
channel ORA_DISK_1: SID=37 device type=DISK

skipping datafile 2; already restored to file /opt/oracle/oradata/FREE/pdbseed/system01.dbf
skipping datafile 4; already restored to file /opt/oracle/oradata/FREE/pdbseed/sysaux01.dbf
skipping datafile 20; already restored to file /opt/oracle/oradata/FREE/pdbseed/undotbs01.dbf
channel ORA_DISK_1: starting datafile backup set restore
channel ORA_DISK_1: specifying datafile(s) to restore from backup set
channel ORA_DISK_1: restoring datafile 00001 to /opt/oracle/oradata/FREE/system01.dbf
channel ORA_DISK_1: restoring datafile 00003 to /opt/oracle/oradata/FREE/sysaux01.dbf
channel ORA_DISK_1: restoring datafile 00007 to /opt/oracle/oradata/FREE/users01.dbf
channel ORA_DISK_1: restoring datafile 00018 to /opt/oracle/oradata/FREE/undotbs01.dbf
channel ORA_DISK_1: reading from backup piece /opt/oracle/oradata/FRA/FREE/backupset/2026_08_28/o1_mf_nnndf_TAG20260828T092124_o92nro55_.bkp
channel ORA_DISK_1: piece handle=/opt/oracle/oradata/FRA/FREE/backupset/2026_08_28/o1_mf_nnndf_TAG20260828T092124_o92nro55_.bkp tag=TAG20260828T092124
channel ORA_DISK_1: restored backup piece 1
channel ORA_DISK_1: restore complete, elapsed time: 00:00:35
channel ORA_DISK_1: starting datafile backup set restore
channel ORA_DISK_1: specifying datafile(s) to restore from backup set
channel ORA_DISK_1: restoring datafile 00021 to /opt/oracle/oradata/FREE/FREEP081/system01.dbf
channel ORA_DISK_1: restoring datafile 00022 to /opt/oracle/oradata/FREE/FREEP081/sysaux01.dbf
channel ORA_DISK_1: restoring datafile 00023 to /opt/oracle/oradata/FREE/FREEP081/undotbs01.dbf
channel ORA_DISK_1: restoring datafile 00024 to /opt/oracle/oradata/FREE/FREEP081/users01.dbf
channel ORA_DISK_1: reading from backup piece /opt/oracle/oradata/FRA/FREE/53103393CA0F0FE9E0636402000A5830/backupset/2026_08_28/o1_mf_nnndf_TAG20260828T092124_o92nsg5v_.bkp
channel ORA_DISK_1: piece handle=/opt/oracle/oradata/FRA/FREE/53103393CA0F0FE9E0636402000A5830/backupset/2026_08_28/o1_mf_nnndf_TAG20260828T092124_o92nsg5v_.bkp tag=TAG20260828T092124
channel ORA_DISK_1: restored backup piece 1
channel ORA_DISK_1: restore complete, elapsed time: 00:00:15
Finished restore at 28-AUG-26
```

Figure 44: SHUTDOWN, STARTUP MOUNT, the SET UNTIL SCN taking effect, and all the datafiles being restored from the compressed backup sets.

The database then has to be opened with RESETLOGS. The redo written after the chosen SCN, which includes the DROP itself, is being deliberately abandoned, so the log sequence cannot continue past a point the datafiles no longer agree with and has to start over.

```
rman target / - RECOVER DATABASE, OPEN RESETLOGS, and the new
incarnation

Starting recover at 28-AUG-26
using channel ORA_DISK_1

starting media recovery
media recovery complete, elapsed time: 00:00:00

Finished recover at 28-AUG-26

Statement processed

List of Database Incarnations
DB Key  Inc Key DB Name  DB ID          STATUS  Reset SCN  Reset Time
-----
1       1       FREE    1506701350     PARENT  1          28-APR-26
2       2       FREE    1506701350     PARENT  2063128    30-MAY-26
3       3       FREE    1506701350     CURRENT 2243876    28-AUG-26

Recovery Manager complete.
```

Figure 45: RECOVER DATABASE, then ALTER DATABASE OPEN RESETLOGS. LIST INCARNATION shows a third incarnation created at SCN 2243876, one past the 2243875 that was asked for, which is the database confirming where it was put back to.

```
sqlplus / as sysdba - the pluggable database has to be opened separately

Pluggable database altered.
```

Figure 46: OPEN RESETLOGS opens the container database and leaves the pluggable databases closed, so FREEPDB1 is opened separately.

Checking the recovery instead of declaring it

The queries from before the backup are now run again, and compared.

```
sqlplus / as sysdba@FREEPDB1 - the same 200,012 rows, and the same
fingerprint

=== the database is open again ===

NAME          LOG_MODE      OPEN_MODE
-----
FREE          ARCHIVELOG    READ WRITE

      CON_ID PDB_NAME      OPEN_MODE
-----
          3 FREEPDB1      READ WRITE

=== the employees table is back ===

TABLE_NAME      ROW_COUNT      SALARY_TOTAL
-----
DEPARTMENTS          10
EMPLOYEES        200,012    15,018,179,225.12

=== and it is the same table, not just a table of the same size ===

FINGERPRINT
-----
430599086091596
```

Figure 47: The same 200,012 rows, the same salary total of 15,018,179,225.12, and the same fingerprint 430599086091596. departments still holds its ten rows, so nothing else was rolled backwards to get the employees back.

```
sqlplus / as sysdba@FREEPDB1 - every object and constraint came back

=== every object, including the two indexes and the function ===

OBJECT_NAME                OBJECT_TYPE  STATUS
-----
DEPT_AVG_SALARY            FUNCTION    VALID
DEPARTMENTS_PK            INDEX       VALID
EMPLOYEES_DEPT_IDX        INDEX       VALID
EMPLOYEES_DEPT_SAL_IDX    INDEX       VALID
EMPLOYEES_PK              INDEX       VALID
DEPARTMENTS               TABLE      VALID
EMPLOYEES                 TABLE      VALID

=== the constraints came back with it ===

TABLE_NAME    CONSTRAINT_NAME    C          STATUS
-----
DEPARTMENTS   SYS_C008648        CHECK      ENABLED
DEPARTMENTS   SYS_C008649        CHECK      ENABLED
DEPARTMENTS   DEPARTMENTS_PK     PRIMARY KEY  ENABLED
EMPLOYEES     SYS_C008651        CHECK      ENABLED
EMPLOYEES     EMPLOYEES_SALARY_CK CHECK      ENABLED
EMPLOYEES     SYS_C008652        CHECK      ENABLED
EMPLOYEES     SYS_C008653        CHECK      ENABLED
EMPLOYEES     EMPLOYEES_PK       PRIMARY KEY  ENABLED
EMPLOYEES     EMPLOYEES_DEPT_FK   FOREIGN KEY  ENABLED
```

Figure 48: Both indexes, the function and every constraint back and valid, including the foreign key to departments and the check on salary. The recovery restored the schema, not just the data.


```
sqlplus / as sysdba@FREEPDB1 - the function runs, and the incarnation moved
on

=== the stored function still runs ===

DEPARTMENT_ID DEPARTMENT_NAME          AVG_SALARY
-----
10 Executive          73,756.54
20 Finance            75,054.12
30 Human Resources    75,330.34
40 Information Technology 74,934.41
50 Operations         75,067.41
60 Retail Banking     75,255.70
70 Treasury           75,014.40
80 Risk and Compliance 75,114.74
90 Internal Audit     74,984.96
100 Customer Service  75,051.49

=== and the incarnation changed, which is the mark of a resetlogs ===

INCARNATION# RESETLOGS_CHANGE# RESETLOGS_TIME          STATUS
-----
1              1 2026-04-28 14:57:23      PARENT
2            2063128 2026-05-30 20:51:50      PARENT
3            2243876 2026-08-28 09:23:27      CURRENT
```

Figure 49: The stored function, INVALID after the drop, recompiling and returning the ten averages again.

	Before the backup	After the recovery
employees rows	200,012	200,012
Sum of salaries	15,018,179,225.12	15,018,179,225.12
Row fingerprint	430599086091596	430599086091596
departments rows	10	10
Objects in HR_DAY9	7 valid	7 valid

The fingerprint matching is the part worth insisting on. Matching row counts would only show that a table of the right size came back. The fingerprint sums a hash of the four columns of every row, so it would change if any single salary came back different, and it did not change.

The optimisations, in one place

Gathered here because the assignment asks for them as explanations rather than as figures.

- An index on employees(department_id). It changes the plan for a query filtering on one department from TABLE ACCESS FULL to INDEX RANGE SCAN, from 1,004 buffer gets to 505, and makes it 10.7x faster. It does nothing at all for the average-per-department query, because that query reads every row by definition.
- A histogram on department_id, gathered with METHOD_OPT FOR COLUMNS SIZE 254. Without it the optimizer assumes the ten departments are equally sized and will not choose the index for any of them. The index is only usable because the statistics describe the skew.
- A covering index on employees(department_id, salary). This is the one that changes the assignment's own query: it becomes an INDEX FAST FULL SCAN with no table access, reading 569 blocks instead of 1,006. It also cuts the single-department query to 4 gets. The measured wall clock for Q1 went up rather than down, and the note in Step 2 explains why.
- Leaving the aggregate in SQL. The PL/SQL formats the result of a GROUP BY rather than fetching rows and averaging them in a loop, which keeps the work in the database instead of moving 200,012 rows across the call interface to compute ten numbers.
- Using INVISIBLE to measure. Not an optimisation of the database, but the reason the before and after numbers are comparable: they were taken against the same segment with the same statistics, with only the optimizer's view of the index changed.

The PL/SQL query used

The cursor at the centre of the anonymous block, quoted from sql/04_plsql.sql. The LEFT JOIN is what makes a department with no employees appear with a NULL average instead of being dropped from the report.

```
CURSOR c_dept_avg IS
  SELECT  d.department_id,
          d.department_name,
          COUNT(e.employee_id) AS headcount,
          AVG(e.salary) AS avg_salary
  FROM    hr_day9.departments d
         LEFT JOIN hr_day9.employees e
              ON e.department_id = d.department_id
  GROUP BY d.department_id, d.department_name
  ORDER BY d.department_id;
```

And the stored function, which returns the same average for one department and is the form the covering index helps most:

```
CREATE OR REPLACE FUNCTION hr_day9.dept_avg_salary (
  p_department_id IN hr_day9.departments.department_id%TYPE
) RETURN NUMBER
IS
  l_avg hr_day9.employees.salary%TYPE;
```

```

BEGIN
    SELECT AVG(salary)
    INTO    l_avg
    FROM    hr_day9.employees
    WHERE   department_id = p_department_id;

    RETURN ROUND(l_avg, 2);
EXCEPTION
    -- AVG over no rows returns NULL rather than raising, so NO_DATA_FOUND
    -- cannot happen here. An invalid department is still worth rejecting
    -- loudly instead of returning a NULL the caller may read as zero.
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20001,
            'cannot average department ' || p_department_id || ': ' || SQLERRM);
END dept_avg_salary;

```

Reproducing it

One script does the whole assignment against a fresh instance and writes every capture the figures are built from. It takes about fifteen minutes, most of it the RMAN backup and restore.

```

cd oracle-tuning
./setup.sh                # instance, schema, tuning, backup, recovery
python3 scripts/make_figures.py # results/ into figures/
python3 build.py          # figures/ into the .docx and .pdf

```

setup.sh runs in a single invocation on purpose. The container runtime on the machine this was built on terminates the container when the shell that started it exits, so splitting the steps across separate shells would lose the database between them.

What I would take away from this

- An index is a claim about a query, not about a column. The assignment's index and the assignment's query look like they belong together, and the execution plan is the thing that says they do not. Reading the plan was the whole difference between reporting an improvement and finding out there was not one.
- A hint that is ignored looks exactly like a hint that worked. Forcing the index on Q1 produced the full scan again, silently, because no legal plan using that index existed. Only the plan output shows the difference.
- Fewer blocks read is not automatically less time. The covering index cut Q1's logical reads by 43 percent and made it slower, because the table already fit in the buffer cache and there was no physical I/O for the saving to save. Collecting both numbers is what caught it.
- PURGE is what turns a dropped table into a real recovery exercise. Without it the recycle bin answers the question and the backup is never tested.
- A recovery is not finished when RMAN says so. The fingerprint, the row counts, the constraints and the untouched departments table are what show that the right point in time was reached and that nothing else moved to get there.